QA Agent Team

How it works, how to install it, and how to use it from Claude desktop

1. What this is

The QA Agent Team is a group of five AI helpers that test a website or an app for you. You point them at a site, and they work out what to test, write the tests, run them, and hand you the results as reports you can read and share.

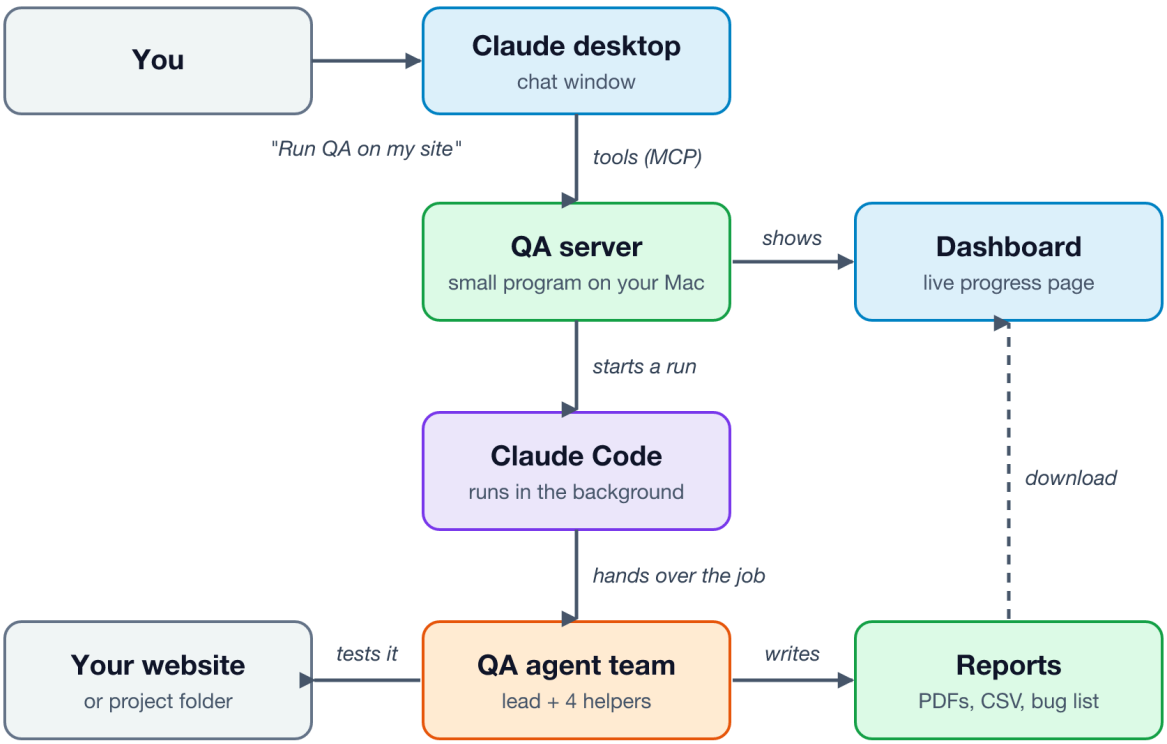
Until now you had to start them from a terminal window. The new interface lets you ask for a test run in an ordinary Claude desktop chat, and watch it happen on a live progress page.

What you get at the end: three PDF reports (functional tests, accessibility, and bugs found) and a spreadsheet file (CSV) of manual tests that you can import into Zephyr Scale.

A real example. We ran it on https://example.com. It took about 12 minutes and cost about $2.56 in Claude usage. It wrote 39 tests, 37 passed, and it found 3 genuine accessibility problems on the site. Larger sites take longer and cost more.

2. How it works

There are several pieces, and each does one simple job. Reading the diagram from the top: you talk to Claude desktop, which talks to a small program on your Mac (the QA server). The QA server starts the real work in the background, and the QA agent team tests your site and writes the reports.

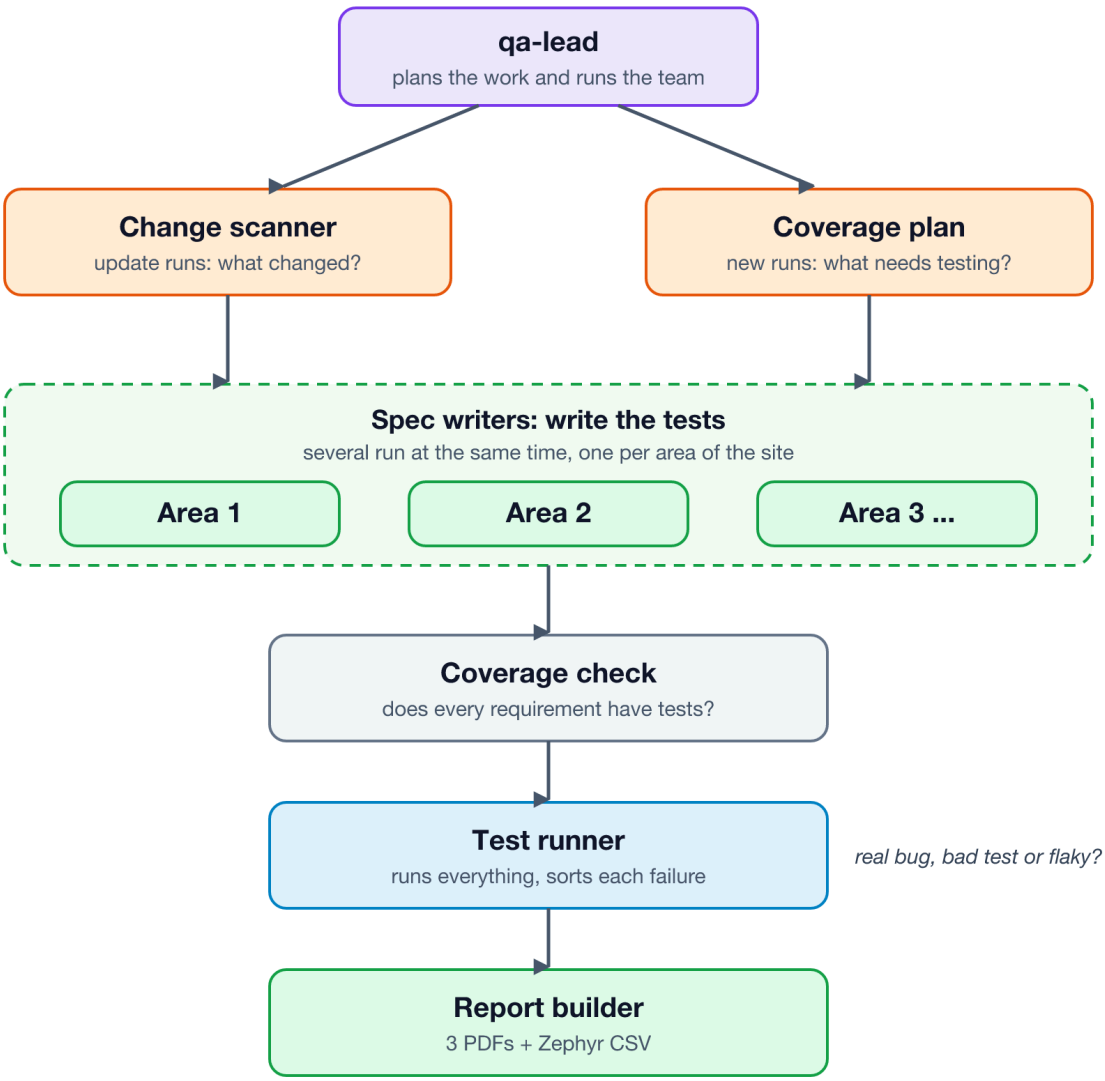


Piece	What it does, in plain English
Claude desktop	**The chat window you already use. You ask for a test run here.**
QA server	A small program that lives in the interface folder. It gives Claude desktop six new abilities (called tools), such as "start a run" and "check progress". It also runs the dashboard.
Dashboard	A web page at http://127.0.0.1:4747 that shows live progress and lets you view or download the reports. It only works on your own Mac. Nobody else can open it.
Claude Code	The command-line version of Claude. The QA server starts it quietly in the background to run the agent team. It needs to be signed in separately from the desktop app.
QA agent team	The five helpers that do the testing (see the next section).
Reports	The PDFs, CSV and bug list the team produces.

A run keeps going on its own. Once started, a run does not depend on the chat window. You can close the chat, restart Claude desktop, and the run carries on. Come back later and ask how it is going.

3. The five helpers

The team follows the same order on every run. One helper (the lead) is in charge and calls the others in turn.



Helper	Its job
qa-lead	**The manager. Looks at your project or site, plans the work, splits it into areas, starts the other helpers, checks the plan is complete, and writes a summary at the end.**
qa-change-scanner	Used for update runs. Works out what has changed since the last full run, so only the new or changed parts need new tests.
qa-spec-writer	Writes the tests for one area of the site. Several run at the same time. They cover normal use, edge cases (such as very long or empty input) and error handling, plus accessibility checks.
qa-test-runner	Runs every test. For each failure it decides: is this a real bug in your site, a mistake in the test, or just flaky? It fixes test mistakes and records real bugs.

Helper	Its job
qa-report-builder	Builds the three PDF reports and the Zephyr CSV, and checks each one looks right.

4. What you get

Everything is saved in a folder called qa, inside the project that was tested. For a website, that project folder is created for you.

File	What it is
qa/reports/qa-report-latest.pdf	**The main test report: coverage percentages, edge-case and error-handling tests, test data and screenshots.**
qa/reports/a11y-report-latest.pdf	Accessibility report, based on the WCAG standard.
qa/reports/bug-report-latest.pdf	The bugs found, with severity, how to reproduce each one, expected versus actual result, and screenshots. Start here.
qa/manual/zephyr-import.csv	Manual tests (things a person must check, like using a screen reader) in the format Zephyr Scale imports.
qa/coverage/COVERAGE.md	A running record of what is covered.
qa/README.md	Instructions for re-running all the tests yourself without Claude.

On the dashboard, every file has a View link. PDFs and the CSV also have a Download button.

5. Installation

What you need first

- A Mac (the steps are the same on other systems, but this guide was tested on a Mac).
- **Claude desktop**, signed in.
- **Claude Code** (the command-line tool). Check it is installed by typing claude --version in Terminal.
- **Node.js 18 or newer**. Check with node --version in Terminal. Nothing else needs installing.
- **Git**, to download the project.

Step by step

1. **Download the project.** In Terminal, run:

```
git clone https://github.com/kranmal/qa-agent-team.git
cd qa-agent-team
```

1. **Install the five helpers.** This copies them into your Claude folder so Claude Code can find them. You only do this once:

```
./install.sh
```

1. **Sign in to Claude Code.** Claude desktop and Claude Code sign in separately, so this step is needed even if the desktop app is already signed in. It opens your browser:

```
claude auth login
```

1. **Connect it to Claude desktop.** This adds the QA server to Claude desktop's settings. It makes a backup of your existing settings first:

```
node interface/setup.mjs --write
```

1. **Restart Claude desktop completely.** Quit it (Cmd+Q on a Mac) and open it again. Closing the window is not enough.
2. **Check it works.** In a new chat, type: Open the QA dashboard. Claude should reply with the address http://127.0.0.1:4747.

Keep the folder where it is. Claude desktop's settings point at the qa-agent-team folder you cloned. If you move or delete it, run step 4 again from the new location.

Want to see what step 4 changes first?

Run it without --write. It only prints the settings entry and changes nothing:

```
node interface/setup.mjs
```

6. Using it

From a Claude desktop chat

Just ask in plain English. Examples:

- Run QA on https://your-site.com
- Run QA on /Users/me/my-app. It runs at http://localhost:3000, start it with npm run dev.
- How is the QA run going?
- Show me the QA reports.
- Open the QA dashboard.
- Stop the QA run.

If your site has a test login, tell Claude in the same message. Never give it a real password.

From the dashboard

Open http://127.0.0.1:4747. Type a website address or project folder, choose Full QA or Update, add any notes, and press Start run. Click any run in the list to see which helpers are working, what they are doing, and the reports as they appear.

When your site changes

Do not start again from scratch. Choose Update (or ask Claude to "scan for changes and update the tests"). The change scanner finds what is new and only the missing tests are written, so it is quicker and cheaper than a full run.

The six tools Claude gets

Tool	What it does
qa_run	**Starts a full or update run on a project folder or a website address. Returns straight away with a run number.**
qa_status	Shows how a run is going: which helpers are working, the latest message, bugs found and files ready.
qa_list_runs	Lists recent runs.
qa_cancel	Stops a run.
qa_reports	Lists the report files for a project, with coverage and bug counts.
qa_open_dashboard	Gives you the dashboard address.

7. Safety, permissions and cost

- **The helpers can run commands without asking.** A background run cannot stop and ask permission, so by default it is pre-approved to read and write files and run shell commands in the project being tested. It needs this to install the test browser and run the tests. Only point it at projects where that is acceptable. For websites, it works in its own separate folder and only reads the site (no logins, no changes).
- **The dashboard is private.** It only listens on your own computer, rejects requests from other websites, and only serves files from a project's qa/reports, qa/manual, qa/coverage and qa/bugs folders.
- **Runs use your Claude usage.** Each run uses the account Claude Code is signed in to. The small example site cost about $2.56. Check the cost on the dashboard when a run finishes.
- **Login check.** If Claude Code is not signed in, starting a run stops straight away with a clear message instead of failing later.

Advanced users can tighten the permissions with the settings QA_UI_ALLOWED_TOOLS and QA_UI_PERMISSION_MODE. They are listed in interface/README.md.

8. Where things are kept

Where	What is there
Your cloned qa-agent-team folder	**The program itself (the interface folder) and the helper definitions.**
~/.claude/agents/	The five helpers, copied there by install.sh.
~/.qa-agent-team-ui/runs/	A record of every run: its progress and output.
~/.qa-agent-team-ui/workspaces/	One folder per website tested. The qa folder with the reports is inside.
Claude desktop settings file	Contains the QA server entry added by step 4. A backup of the old file was saved next to it.

9. If something goes wrong

Problem	What to do
The QA tools don’t appear in Claude desktop	**Run node interface/setup.mjs --write again, then quit and reopen Claude desktop (Cmd+Q, not just closing the window).**
"Claude Code CLI is not logged in"	Run claude auth login in Terminal and sign in. The desktop app’s sign-in is separate.
"qa-lead agent not found"	Run ./install.sh from the qa-agent-team folder.
A run fails straight away	Open the run on the dashboard and read the error message. It is almost always a sign-in problem.
Tests fail in the report	This is normal. Real bugs are meant to fail. Open the bug report first.
The dashboard won’t open	Another copy may already be using the address. Run node interface/server.mjs --web and use the address it prints. Or ask Claude "Open the QA dashboard".
Reports look out of date	Ask Claude to re-run everything and rebuild the reports.

Problem	What to do
You want to remove it	Delete the qa-agent-team entry from Claude desktop’s settings file (or restore the backup), then delete the folders listed in section 8.